

A hybrid matrix-free solver for the convection–diffusion–reaction equation

Junkai Ji Emiliano Liao

Numerical Methods for Partial Differential Equations

Abstract. We implement a matrix-free finite element solver for the convection–diffusion–reaction problem in deal.II, preconditioned by a geometric multigrid V-cycle running in single precision on the levels, and parallelised on three levels: MPI across a distributed mesh, TBB across cell batches, and SIMD lanes inside a batch. An assembled sparse-matrix solver of the same bilinear form is included as a reference. The two backends agree on the discrete solution to the solver tolerance. The matrix-free operator uses $16.4\times$ less memory than the assembled matrix at 3D Q_3 and applies it $6\times$ faster, while the assembled matrix wins at low order. Geometric multigrid needs a mesh-independent 4–5 iterations where algebraic multigrid on the assembled matrix needs up to 44. The parallel scaling measured here is limited by the test machine, a 4+4 core laptop, and we report it as such.

1 The problem

We solve the convection–diffusion–reaction equation on $\Omega = (0, 1)^d$,

$$-\nabla \cdot (\mu \nabla u) + \beta \cdot \nabla u + \gamma u = f \quad \text{in } \Omega,$$

with $u = g$ on Γ_D and $\mu \nabla u \cdot \mathbf{n} = h$ on Γ_N , where Γ_N is one face of the cube and Γ_D is the rest of the boundary. The data are manufactured from

$$u(\mathbf{x}) = \prod_d \cos(\pi x_d / 2),$$

which is an eigenfunction of the Laplacian, $-\Delta u = d(\pi/2)^2 u$, so f follows in closed form. The solution is non-zero on the faces $x_d = 0$, so the Dirichlet data are inhomogeneous, and its normal derivative on the Neumann face is non-zero, so both boundary conditions are exercised rather than being silently trivial.

2 Discretisation and solver

The linear system is solved with FGMRES (restart 50) to a relative residual tolerance of 10^{-9} . FGMRES rather than GMRES because the multigrid preconditioner runs in single precision and is therefore not a fixed linear operator in the outer double-precision iteration.

Three preconditioners are available for each backend, so that the multigrid result is reported against a baseline rather than on its own: none, a Jacobi sweep with the operator diagonal, and multigrid.

2.1 The matrix-free operator

The operator is never assembled. Each application evaluates

$$a(u, v) = (\mu \nabla u, \nabla v) + (\beta \cdot \nabla u, v) + (\gamma u, v)$$

cell batch by cell batch through `FEEvaluation`, with the quadrature-point kernel shared between the operator application, the diagonal computation and the right-hand-side assembly, so that the three cannot drift apart. Only the mesh geometry and the vectors are stored; the element matrices never exist.

2.2 Geometric multigrid

The preconditioner is a V-cycle over the mesh hierarchy. Level matrices, transfers and smoothers all run in single precision, with the conversion at the fine-grid interface, which halves the memory traffic of the part of the solver that dominates the run time.

2.3 The matrix-based reference

To make the comparison meaningful the reference is not a different method but the same one written the conventional way: the same bilinear form, the same Gauss quadrature, the same FGMRES and the same tolerance, differing only in that the operator is assembled into a distributed Trilinos sparse matrix and applied as a sparse matrix–vector product. Inhomogeneous Dirichlet data are lifted by the constraints during assembly instead of by the correction solve the matrix-free path uses; both reach the same discrete solution, which Section 3 checks. Its multigrid option is algebraic multigrid, the natural counterpart of the geometric V-cycle for an assembled matrix, with the non-symmetric smoother selected.

2.4 Hybrid parallelism

Three levels of parallelism act on the cell loop at once. MPI owns a `parallel::distributed::Triangulation` partitioned by p4est. Within a rank, TBB splits the local cell batches under deal.II's `partition_partition` scheme. Within a batch, `VectorizedArray` fills the SIMD lanes, two doubles wide on the ARM NEON unit of the test machine.

3 Verification

Table 1 lists the errors of both backends on the same sequence of meshes with Q_2 elements. The observed orders are the optimal $p + 1$ in L^2 and p in H^1 .

Cells	DoFs	L^2 (MF)	L^2 (MB)	rate	H^1 (MF)	rate
64	289	3.08e−5	3.08e−5	—	1.60e−3	—
256	1089	3.85e−6	3.85e−6	3.00	3.99e−4	2.00
1024	4225	4.81e−7	4.81e−7	3.00	9.97e−5	2.00
4096	16641	6.01e−8	6.02e−8	3.00	2.49e−5	2.00
16384	66049	7.52e−9	8.02e−9	3.00	6.23e−6	2.00

Table 1: Convergence of both backends, 2D Q_2 . MF is matrix-free, MB is matrix-based. The two columns of L^2 errors agree to the digits shown except in the last, where they differ by less than the solver tolerance.

The agreement between the two error columns is the cross-check that matters: the matrix-free quadrature kernel and the assembled element matrices were written independently, and they discretise the same problem. A sign error or a missing term in either would show up here as a broken convergence rate.

The unit tests carry the rest of the verification: that the operator reduces to zero on constants when convection and reaction are switched off, that it is symmetric without convection and provably not symmetric with it, that the form is affine in (μ, γ, β) , and that the V-cycle is linear and keeps the iteration count bounded under refinement.

4 Matrix-based against matrix-free

All runs in this section use one MPI rank and one thread, so that the operator representation is the only variable.

4.1 Memory

Dim	Degree	DoFs	Matrix-free	Matrix-based	Ratio
2 D	Q_1	1050625	244.7 MB	111.8 MB	$0.46\times$
2 D	Q_2	1050625	100.5 MB	195.4 MB	$1.94\times$
2 D	Q_3	591361	39.8 MB	170.2 MB	$4.28\times$
3 D	Q_1	274625	85.3 MB	77.3 MB	$0.91\times$
3 D	Q_2	274625	27.7 MB	177.2 MB	$6.41\times$
3 D	Q_3	117649	8.5 MB	138.6 MB	$16.39\times$

Table 2: Operator memory, summed over ranks. Matrix-free includes the **MatrixFree** bookkeeping and every multigrid level; matrix-based is the sparse matrix and its sparsity pattern. Ratio above one favours matrix-free.

The trend in Table 2 is the expected one and is the main argument for the method. The assembled matrix stores $O(p^d)$ entries per row, so its footprint grows quickly with the polynomial degree and the dimension, while the matrix-free operator stores only geometry and vectors. At the lowest order the comparison reverses: a Q_1 matrix is cheap, and the **MatrixFree** tables plus the multigrid levels cost more than it does.

4.2 Operator application

Dim	Degree	Matrix-free	Matrix-based	Ratio	
2 D	Q_1	29.6	210.2	$0.14\times$	
2 D	Q_2	71.8	125.7	$0.57\times$	
2 D	Q_3	95.7	69.7	$1.37\times$	
3 D	Q_1	17.0	75.3	$0.23\times$	
3 D	Q_2	49.2	23.6	$2.08\times$	
3 D	Q_3	68.5	11.3	$6.05\times$	

Table 3: Throughput of a bare operator application in MDoF/s, averaged over 50 applications after a warm-up. The bar shows the matrix-free figure. Ratio above one favours matrix-free.

Table 3 shows the crossover clearly. At Q_1 the sparse matrix–vector product is several times faster: the matrix has few entries per row and the hardware is very good at streaming them, whereas matrix-free pays for sum-factorisation it cannot amortise over so few degrees of freedom per cell. By 3D Q_3 the ranking has reversed and matrix-free is several times ahead, because the assembled matrix has become bandwidth-bound while the matrix-free kernel does the same work out of cache.

Both figures are low in absolute terms. A matrix-free application of this kind is memory-bandwidth bound, and the test machine is a laptop.

4.3 Iteration count and time to solution

Dim	Degree	GMG its	AMG its	Assembly (MB)	Solve (MF)	Solve (MB)
2 D	Q_1	5	15	0.61 s	1.82 s	2.19 s
2 D	Q_2	4	29	0.62 s	0.68 s	3.49 s
2 D	Q_3	4	44	0.52 s	0.28 s	3.42 s
3 D	Q_1	5	19	0.60 s	0.76 s	1.04 s
3 D	Q_2	4	24	1.29 s	0.23 s	2.38 s
3 D	Q_3	5	41	1.86 s	0.08 s	2.51 s

Table 4: Iterations and wall times. GMG is the geometric V-cycle used by the matrix-free backend, AMG the algebraic multigrid used by the matrix-based one. The assembly column is the cost the matrix-free backend does not pay at all.

The geometric V-cycle holds 4–5 iterations across every problem in Table 4, independent of mesh size and degree, which is what a well-tuned multigrid should do. Algebraic multigrid on the same problems needs between 15 and 44 iterations and degrades with the polynomial degree, since the coarsening heuristics see only the matrix graph and not the mesh hierarchy that produced it.

The assembly column is worth separating out. It is paid on every mesh, before the solve begins, and it is absent from the matrix-free path, where the equivalent right-hand-side assembly costs 0.010 s against 1.86 s at 3D Q_3 .

5 MPI strong scaling

Table 5 fixes a 3D Q_2 problem of 2146689 degrees of freedom and varies the rank count at one thread per rank.

Ranks	Solve	Speedup	Efficiency	MDoF/s	
1	1.98 s	1.00×	100%	43.5	
2	1.54 s	1.28×	64%	59.7	
4	2.81 s	0.70×	18%	45.5	
8	23.02 s	0.09×	1%	23.9	

Table 5: Strong scaling of the matrix-free backend, 3D Q_2 , one thread per rank. Speedup is against the single-rank solve time. The bar is drawn against an ideal speedup of 8.

Scaling is poor and turns negative. Two ranks give a modest gain; beyond that the solve slows down, and at eight ranks it is far slower than serial.

The machine explains most of this. The test system is an Apple M1 with four performance and four efficiency cores. Once more than four ranks are active, some of them are scheduled onto efficiency cores that are several times slower. The multigrid V-cycle synchronises on every level, so the slowest rank sets the pace for all of them, and the coarse levels have too few cells per rank to amortise the communication. A run on homogeneous cores would not show this collapse, and this measurement should be repeated on a cluster node before any conclusion is drawn about the implementation.

6 Hybrid scaling

Table 6 crosses ranks with threads per rank on the same problem, never exceeding the eight cores of the machine.

Ranks	Threads	Workers	Solve	MDoF/s	
1	1	1	2.17 s	39.4	
1	2	2	2.16 s	39.0	
1	4	4	2.25 s	38.7	
1	8	8	2.13 s	42.0	
2	1	2	1.66 s	56.3	
2	2	4	1.61 s	53.8	
2	4	8	1.64 s	55.4	
4	1	4	2.90 s	41.1	
4	2	8	2.94 s	42.1	
8	1	8	22.77 s	22.7	

Table 6: Hybrid grid, 3D Q_2 . Workers is ranks $\times$ threads. The bar is the speedup of the solve against the one rank, one thread run, drawn against an ideal of 8.

The thread axis is flat. Holding the rank count at one and going from one to eight TBB threads changes the operator throughput from 39.4 to 42.0 MDoF/s, a few percent rather than the several-fold gain the `partition_partition` scheme is supposed to deliver. The same flatness appears at two and four ranks.

We checked the obvious explanations and none of them account for it. `deal.II` is built with TBB support; the thread limit is applied and the solver reports the requested thread count; removing the workaround that pins Kokkos to a single thread changes nothing; and the problem is large enough that the cell batches are not exhausted. Measuring CPU time against wall time gives a ratio of about 1.5 at four threads where four busy cores would give close to 4, so the cell loop is genuinely not being spread across the threads. The cause is still open, and until it is found the thread level of the hybrid parallelism should be regarded as present in the code but unverified in practice.

7 Limitations

The parallel measurements are the weak part of this report and we would rather state that plainly than dress them up. They were taken on a single laptop with heterogeneous cores, which is the wrong instrument for a scaling study: it cannot show whether the MPI collapse past four ranks is a property of the implementation or of the scheduler putting ranks on efficiency cores, and its memory bandwidth caps every throughput figure here. The threading result is a genuine open defect rather than a hardware artefact.

The single-rank comparison of Table 2, Table 3 and Table 4 does not suffer from this. Those runs are serial, and the crossover they show between the two operator representations is the expected one and is consistent across six combinations of dimension and degree.